

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Code Challenge (High)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5
9	Implement a sparse index on a sorted file in code and compare its search	BL4 – Analyze	5

	performance with a dense index using timing analysis.		
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

*I **RAVURU PRANATHI (192525076)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

CODE:

```
# Heap File Organization Simulation

heap_file = []

def insert_record():

    record = input("Enter record to insert: ")

    heap_file.append(record)

    print("Record inserted successfully.")

def delete_record():

    record = input("Enter record to delete: ")

    if record in heap_file:

        heap_file.remove(record)

        print("Record deleted successfully.")

    else:

        print("Record not found.")

def sequential_scan():

    if len(heap_file) == 0:

        print("Heap file is empty.")

    else:

        print("\nRecords in Heap File:")

        for i, record in enumerate(heap_file, start=1):

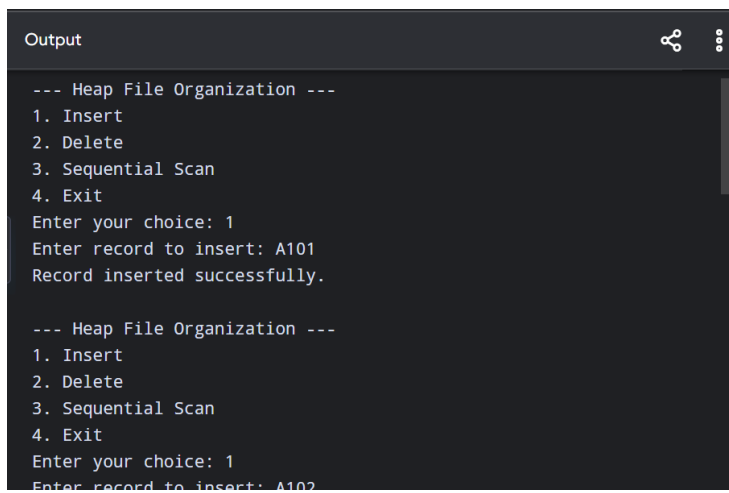
            print(f"Record {i}: {record}")

while True:

    print("\n--- Heap File Organization ---")
```

```
print("1. Insert")
print("2. Delete")
print("3. Sequential Scan")
print("4. Exit")
choice = int(input("Enter your choice: "))
if choice == 1:
    insert_record()
elif choice == 2:
    delete_record()
elif choice == 3:
    sequential_scan()
elif choice == 4:
    print("Program terminated.")
    break
else:
    print("Invalid choice.")
```

INPUT & OUTPUT:

A screenshot of a terminal window with a dark background. The window title is "Output". It shows the program's execution flow: a menu with four options (1. Insert, 2. Delete, 3. Sequential Scan, 4. Exit), followed by user input "1", then "Enter record to insert: A101", and the output "Record inserted successfully.". This sequence repeats once more, showing the menu again and the start of the next input sequence.

```
Output
--- Heap File Organization ---
1. Insert
2. Delete
3. Sequential Scan
4. Exit
Enter your choice: 1
Enter record to insert: A101
Record inserted successfully.

--- Heap File Organization ---
1. Insert
2. Delete
3. Sequential Scan
4. Exit
Enter your choice: 1
Enter record to insert: A102
```

```
Output
Enter your choice: 1
Enter record to insert: A102
Record inserted successfully.

--- Heap File Organization ---
1. Insert
2. Delete
3. Sequential Scan
4. Exit
Enter your choice: 1
Enter record to insert: A103
Record inserted successfully.

--- Heap File Organization ---
1. Insert
2. Delete
3. Sequential Scan
```

```
Output
Enter your choice: 3

Records in Heap File:
Record 1: A101
Record 2: A102
Record 3: A103
|
--- Heap File Organization ---
1. Insert
2. Delete
3. Sequential Scan
4. Exit
Enter your choice: 2
Enter record to delete: A101
Record deleted successfully.
```

```
Output
2. Delete
3. Sequential Scan
4. Exit
Enter your choice: 2
Enter record to delete: A101
Record deleted successfully.

--- Heap File Organization ---
1. Insert
2. Delete
3. Sequential Scan
4. Exit
Enter your choice: 4
Program terminated.

=== Code Execution Successful ===
```

2. Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

CODE:

```
# Static Hashing with Chaining

# Student Database - 500 Records

TABLE_SIZE = 50

# Create hash table with empty chains
hash_table = [[] for _ in range(TABLE_SIZE)]

# Hash function
def hash_function(student_id):
    return student_id % TABLE_SIZE

# Insert student
def insert(student_id, name):
    index = hash_function(student_id)
    hash_table[index].append((student_id, name))
    print("Student record inserted successfully.")

# Search student
def search(student_id):
    index = hash_function(student_id)

    for record in hash_table[index]:
        if record[0] == student_id:
            print("Student Found")
            print("ID :", record[0])
            print("Name :", record[1])
```

```

        return

    print("Student not found.")

# Display hash table

def display():

    print("\n--- Hash Table ---")

    for i in range(TABLE_SIZE):

        print(i, ":", end=" ")

        if len(hash_table[i]) == 0:

            print("Empty")

        else:

            for record in hash_table[i]:

                print(f"[{record[0]}, {record[1]}", end=" -> ")

            print("NULL")

# Main program

while True:

    print("\n--- Student Database ---")

    print("1. Insert Student")

    print("2. Search Student")

    print("3. Display Hash Table")

    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:

        student_id = int(input("Enter Student ID: "))

        name = input("Enter Student Name: ")

        insert(student_id, name)

```

```
elif choice == 2:

    student_id = int(input("Enter Student ID to search: "))

    search(student_id)

elif choice == 3:

    display()

elif choice == 4:

    print("Program terminated.")

    break

else:

    print("Invalid choice.")
```

INPUT & OUTPUT:

```
Output
--- Student Database ---
1. Insert Student
2. Search Student
3. Display Hash Table
4. Exit
Enter your choice: 1
Enter Student ID: 101
Enter Student Name: Rahul
Student record inserted successfully.

--- Student Database ---
1. Insert Student
2. Search Student
3. Display Hash Table
4. Exit
```



```
Enter your choice: 1
Enter Student ID: 151
Enter Student Name: Priya
Student record inserted successfully.
```

```
--- Student Database ---
```

1. Insert Student
2. Search Student
3. Display Hash Table
4. Exit

```
Enter your choice: 1
Enter Student ID: 201
Enter Student Name: Rani
Student record inserted successfully.
```

Output

```
--- Student Database ---
```

1. Insert Student
2. Search Student
3. Display Hash Table
4. Exit

```
Enter your choice: 3
```

```
--- Hash Table ---
```

```
0 : Empty
1 : [101, Rahul] -> [151, Priya] -> [201, Rani] -> NULL
2 : Empty
3 : Empty
4 : Empty
5 : Empty
6 : Empty
7 : Empty
```

Output

```
8 : Empty
9 : Empty
10 : Empty
11 : Empty
12 : Empty
13 : Empty
14 : Empty
15 : Empty
16 : Empty
17 : Empty
18 : Empty
19 : Empty
20 : Empty
21 : Empty
22 : Empty
23 : Empty
```

```
Output
29 : Empty
30 : Empty
31 : Empty
32 : Empty
33 : Empty
34 : Empty
35 : Empty
36 : Empty
37 : Empty
38 : Empty
39 : Empty
40 : Empty
41 : Empty
42 : Empty
43 : Empty
44 : Empty
```

```
Output
44 : Empty
45 : Empty
46 : Empty
47 : Empty
48 : Empty
49 : Empty

--- Student Database ---
1. Insert Student
2. Search Student
3. Display Hash Table
4. Exit
Enter your choice: 4
Program terminated.

=== Code Execution Successful ===
```

3. Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

CODE:

```
# Sorted File Organization with Binary Search

records = []

def insert_record():

    student_id = int(input("Enter Student ID: "))

    name = input("Enter Student Name: ")

    records.append((student_id, name))

    # Keep records sorted by primary key

    records.sort(key=lambda x: x[0])

    print("Record inserted successfully.")

def display_records():

    if not records:

        print("No records available.")

        return

    print("\n--- Sorted Records ---")

    for record in records:

        print("Student ID:", record[0], "Name:", record[1])

def binary_search():

    key = int(input("Enter Student ID to search: "))

    low = 0

    high = len(records) - 1

    while low <= high:

        mid = (low + high) // 2
```

```
    if records[mid][0] == key:
        print("\nRecord Found")
        print("Student ID:", records[mid][0])
        print("Student Name:", records[mid][1])
        return
    elif records[mid][0] < key:
        low = mid + 1
    else:
        high = mid - 1
print("Record not found.")
while True:
    print("\n--- Sorted File Organization ---")
    print("1. Insert Record")
    print("2. Display Records")
    print("3. Binary Search")
    print("4. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        insert_record()
    elif choice == 2:
        display_records()
    elif choice == 3:
        binary_search()
    elif choice == 4:
        print("Program terminated.")
```

break

else:

print("Invalid choice.")

INPUT & OUTPUT:

```
Output
--- Sorted File Organization ---
1. Insert Record
2. Display Records
3. Binary Search
4. Exit
Enter your choice: 1
Enter Student ID: 101
Enter Student Name: Rahul
Record inserted successfully.

--- Sorted File Organization ---
1. Insert Record
2. Display Records
3. Binary Search
4. Exit
Enter your choice: 1
```

```
Output
Enter your choice: 1
Enter Student ID: 102
Enter Student Name: Pria
Record inserted successfully.

--- Sorted File Organization ---
1. Insert Record
2. Display Records
3. Binary Search
4. Exit
Enter your choice: 1
Enter Student ID: 103
Enter Student Name: Paya
Record inserted successfully.

--- Sorted File Organization ---
```

```
Output
--- Sorted File Organization ---
1. Insert Record
2. Display Records
3. Binary Search
4. Exit
Enter your choice: 2

--- Sorted Records ---
Student ID: 101 Name: Rahul
Student ID: 102 Name: Pria
Student ID: 103 Name: Paya

--- Sorted File Organization ---
1. Insert Record
2. Display Records
3. Binary Search
```

```
Output
4. Exit
Enter your choice: 3
Enter Student ID to search: 103

Record Found
Student ID: 103
Student Name: Paya

--- Sorted File Organization ---
1. Insert Record
2. Display Records
3. Binary Search
4. Exit
Enter your choice: 4
Program terminated.
```

4. Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.

CODE:

```
# B-Tree of Order 4

# Maximum 3 keys per node

class BTreeNode:

    def __init__(self, leaf=False):

        self.keys = []

        self.children = []

        self.leaf = leaf

class BTree:

    def __init__(self):

        self.root = BTreeNode(True)

    # Search a key

    def search(self, node, key):

        i = 0

        while i < len(node.keys) and key > node.keys[i]:

            i += 1

        if i < len(node.keys) and key == node.keys[i]:

            return True

        if node.leaf:

            return False

        return self.search(node.children[i], key)

    # Split a full child

    def split_child(self, parent, index):
```

```

full_child = parent.children[index]
new_child = BTreeNode(full_child.leaf)
# Middle key moves to parent
middle_key = full_child.keys[1]
# Move right key to new child
new_child.keys = full_child.keys[2:]
# Keep left key in original child
full_child.keys = full_child.keys[:1]
# Move children if not leaf
if not full_child.leaf:
    new_child.children = full_child.children[2:]
    full_child.children = full_child.children[:2]
parent.children.insert(index + 1, new_child)
parent.keys.insert(index, middle_key)
# Insert key
def insert(self, key):
    root = self.root
    # If root is full
    if len(root.keys) == 3:
        new_root = BTreeNode(False)
        new_root.children.append(root)
        self.root = new_root
        self.split_child(new_root, 0)
        self.insert_non_full(new_root, key)
    else:

```



```

        self.insert_non_full(root, key)
# Insert into a non-full node
def insert_non_full(self, node, key):
    i = len(node.keys) - 1
    if node.leaf:
        node.keys.append(0)
        while i >= 0 and key < node.keys[i]:
            node.keys[i + 1] = node.keys[i]
            i -= 1
        node.keys[i + 1] = key
    else:
        while i >= 0 and key < node.keys[i]:
            i -= 1
        i += 1
        # Split child if full
        if len(node.children[i].keys) == 3:
            self.split_child(node, i)
            if key > node.keys[i]:
                i += 1
            self.insert_non_full(node.children[i], key)
# Display B-Tree
def display(self, node=None, level=0):
    if node is None:
        node = self.root
    print("Level", level, ":", node.keys)

```

```

        if not node.leaf:
            for child in node.children:
                self.display(child, level + 1)

# Main program
btree = BTree()

keys = [10, 20, 5, 6, 12, 30, 7, 17]

print("Keys inserted:")

print(keys)

for key in keys:
    btree.insert(key)

print("\n--- B-Tree ---")

btree.display()

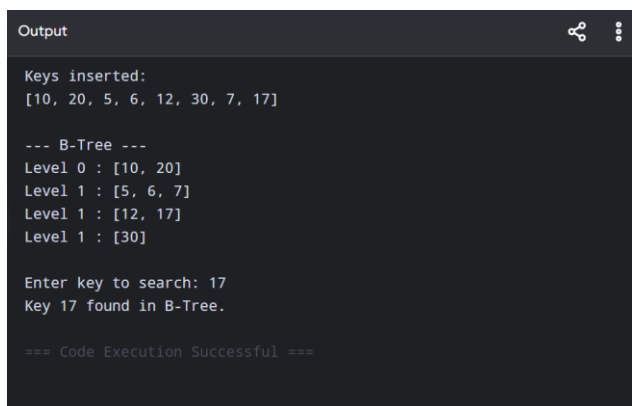
# Search operation

key = int(input("\nEnter key to search: "))

if btree.search(btree.root, key):
    print("Key", key, "found in B-Tree.")
else:
    print("Key", key, "not found in B-Tree.")

```

INPUT & OUPUT:



The screenshot shows a terminal window titled "Output" with the following text:

```

Keys inserted:
[10, 20, 5, 6, 12, 30, 7, 17]

--- B-Tree ---
Level 0 : [10, 20]
Level 1 : [5, 6, 7]
Level 1 : [12, 17]
Level 1 : [30]

Enter key to search: 17
Key 17 found in B-Tree.

=== Code Execution Successful ===

```

5. Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.

CODE:

```
# B+ Tree Implementation

# Integer Keys with Leaf-Level Linking

ORDER = 4

class Node:

    def __init__(self, leaf=False):

        self.keys = []

        self.children = []

        self.leaf = leaf

        self.next = None

class BPlusTree:

    def __init__(self):

        self.root = Node(leaf=True)

    # Insert a key

    def insert(self, key):

        root = self.root

        # If root is full, split it

        if len(root.keys) == ORDER - 1:

            new_root = Node()

            new_root.children.append(root)

            self.root = new_root

            self.split_child(new_root, 0)

            self.insert_non_full(new_root, key)
```

```

else:
    self.insert_non_full(root, key)
# Split a child
def split_child(self, parent, index):
    child = parent.children[index]
    new_node = Node(leaf=child.leaf)
    if child.leaf:
        # Split leaf
        mid = len(child.keys) // 2
        new_node.keys = child.keys[mid:]
        child.keys = child.keys[:mid]
        # Link leaves
        new_node.next = child.next
        child.next = new_node
        # First key of new leaf goes to parent
        parent.keys.insert(index, new_node.keys[0])
        parent.children.insert(index + 1, new_node)
    else:
        # Split internal node
        mid = len(child.keys) // 2
        promoted_key = child.keys[mid]
        new_node.keys = child.keys[mid + 1:]
        child.keys = child.keys[:mid]
        new_node.children = child.children[mid + 1:]
        child.children = child.children[:mid + 1]

```

```

        parent.keys.insert(index, promoted_key)

        parent.children.insert(index + 1, new_node)

# Insert into non-full node

def insert_non_full(self, node, key):
    if node.leaf:
        node.keys.append(key)
        node.keys.sort()
    else:
        i = len(node.keys) - 1
        while i >= 0 and key < node.keys[i]:
            i -= 1
        i += 1
        # Split child if full
        if len(node.children[i].keys) == ORDER - 1:
            self.split_child(node, i)
            if key >= node.keys[i]:
                i += 1
            self.insert_non_full(node.children[i], key)

# Search for a key

def search(self, key):
    node = self.root
    while not node.leaf:
        i = 0
        while i < len(node.keys) and key >= node.keys[i]:
            i += 1

```

```

        node = node.children[i]

    return key in node.keys

# Range search using linked leaves
def range_search(self, start, end):
    node = self.root

    # Find first leaf
    while not node.leaf:
        i = 0

        while i < len(node.keys) and start >= node.keys[i]:
            i += 1

        node = node.children[i]

    result = []

    # Follow linked leaves
    while node:
        for key in node.keys:
            if start <= key <= end:
                result.append(key)

            if key > end:
                return result

        node = node.next

    return result

# Display tree
def display(self, node=None, level=0):
    if node is None:
        node = self.root

```

```

        print("Level", level, ":", node.keys)
    if not node.leaf:
        for child in node.children:
            self.display(child, level + 1)
# Display leaf links
def display_leaf_links(self):
    node = self.root
    while not node.leaf:
        node = node.children[0]
    print("\nLeaf-level linked list:")
    while node:
        print(node.keys, end=" -> ")
        node = node.next
    print("NULL")
# Main program
bpt = BPlusTree()
keys = [10, 20, 5, 6, 12, 30, 7, 17, 25, 35]
print("Keys inserted:")
print(keys)
for key in keys:
    bpt.insert(key)
print("\n--- B+ Tree ---")
bpt.display()
bpt.display_leaf_links()
# Search

```

```
key = int(input("\nEnter key to search: "))
```

```
if bpt.search(key):
```

```
    print("Key", key, "found.")
```

```
else:
```

```
    print("Key", key, "not found.")
```

```
# Range query
```

```
start = int(input("\nEnter range start: "))
```

```
end = int(input("Enter range end: "))
```

```
result = bpt.range_search(start, end)
```

```
print("Keys in range:", result)
```

INPUT & OUTPUT:

```
Output
Keys inserted:
[10, 20, 5, 6, 12, 30, 7, 17, 25, 35]

--- B+ Tree ---
Level 0 : [12]
Level 1 : [10]
Level 2 : [5, 6, 7]
Level 2 : [10]
Level 1 : [20, 25]
Level 2 : [12, 17]
Level 2 : [20]
Level 2 : [25, 30, 35]

Leaf-level linked list:
[5, 6, 7] -> [10] -> [12, 17] -> [20] -> [25, 30, 35] -> NULL
```

```
Output
Level 1 : [20, 25]
Level 2 : [12, 17]
Level 2 : [20]
Level 2 : [25, 30, 35]

Leaf-level linked list:
[5, 6, 7] -> [10] -> [12, 17] -> [20] -> [25, 30, 35] -> NULL

Enter key to search: 17
Key 17 found.

Enter range start: 7
Enter range end: 25
Keys in range: [7, 10, 12, 17, 20, 25]

=== Code Execution Successful ===
```


6. Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

CODE:

```
# Dense Primary Index on Sorted File

# Sorted file: (Primary Key, Student Name)

records = [

    (101, "Ayesha"),

    (103, "Rahul"),

    (105, "Priya"),

    (107, "Arun"),

    (109, "Sneha"),

    (111, "Kiran")

]

# Create Dense Primary Index

# Every record has an entry in the index

index = []

for position, record in enumerate(records):

    index.append((record[0], position))

# Display index table

def display_index():

    print("\n--- Dense Primary Index ---")

    print("Index Key\tRecord Position")

    for key, position in index:

        print(key, "\t\t", position)

# Search using index
```

```

def search(key):

    for index_key, position in index:

        if index_key == key:

            print("\nRecord Found")

            print("Primary Key :", records[position][0])

            print("Student Name:", records[position][1])

            return

        print("\nRecord not found.")

# Main program

while True:

    print("\n--- Dense Primary Index ---")

    print("1. Display Index")

    print("2. Search Record")

    print("3. Display Sorted File")

    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:

        display_index()

    elif choice == 2:

        key = int(input("Enter primary key to search: "))

        search(key)

    elif choice == 3:

        print("\n--- Sorted File ---")

        for record in records:

            print(record[0], record[1])

```

```

elif choice == 4:

    print("Program terminated.")

    break

else:

    print("Invalid choice.")

```

INPUT & OUTPUT:

```

Output
--- Dense Primary Index ---
1. Display Index
2. Search Record
3. Display Sorted File
4. Exit
Enter your choice: 1

--- Dense Primary Index ---
Index Key    Record Position
101          0
103          1
105          2
107          3
109          4
111          5

```

```

Output
--- Dense Primary Index ---
1. Display Index
2. Search Record
3. Display Sorted File
4. Exit
Enter your choice: 2
Enter primary key to search: 105

Record Found
Primary Key : 105
Student Name: Priya

--- Dense Primary Index ---
1. Display Index
2. Search Record
3. Display Sorted File

```

```
4. Exit
Enter your choice: 3
```

```
--- Sorted File ---
```

```
101 Ayesha
103 Rahul
105 Priya
107 Arun
109 Sneha
111 Kiran
```

```
--- Dense Primary Index ---
```

```
1. Display Index
2. Search Record
3. Display Sorted File
4. Exit
```

Output

```
101 Ayesha
103 Rahul
105 Priya
107 Arun
109 Sneha
111 Kiran
```

```
--- Dense Primary Index ---
```

```
1. Display Index
2. Search Record
3. Display Sorted File
4. Exit
```

```
Enter your choice: 4
```

```
Program terminated.
```

```
=== Code Execution Successful ===
```

ENG

11:44 PM

7. Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.

CODE:

```
# Extendible Hashing

# Demonstrates Directory Doubling

# Inserting 8 Records

BUCKET_SIZE = 2

class Bucket:

    def __init__(self, depth):

        self.depth = depth

        self.records = []

class ExtendibleHashing:

    def __init__(self):

        self.global_depth = 1

        bucket0 = Bucket(1)

        bucket1 = Bucket(1)

        self.directory = [bucket0, bucket1]

    def hash_function(self, key):

        # Use the last bits of the key

        return key & ((1 << self.global_depth) - 1)

    def double_directory(self):

        old_directory = self.directory

        self.directory = old_directory + old_directory

        self.global_depth += 1

        print("\n*** Directory Doubled ***")

        print("Global Depth:", self.global_depth)
```

```

def split_bucket(self, bucket_index):

    old_bucket = self.directory[bucket_index]

    old_depth = old_bucket.depth

    new_depth = old_depth + 1

    # Create two new buckets

    bucket0 = Bucket(new_depth)

    bucket1 = Bucket(new_depth)

    # Redistribute records

    for key in old_bucket.records:

        bit = (key >> old_depth) & 1

        if bit == 0:

            bucket0.records.append(key)

        else:

            bucket1.records.append(key)

    # Update bucket depth

    old_bucket.depth = new_depth

    # Update directory pointers

    for i in range(len(self.directory)):

        if self.directory[i] is old_bucket:

            bit = (i >> old_depth) & 1

            if bit == 0:

                self.directory[i] = bucket0

            else:

                self.directory[i] = bucket1

def insert(self, key):

```

```

while True:

    index = self.hash_function(key)

    bucket = self.directory[index]

    if key in bucket.records:

        print("Duplicate key:", key)

        return

    if len(bucket.records) < BUCKET_SIZE:

        bucket.records.append(key)

        print("Inserted:", key)

        return

    # Bucket overflow

    print("\nOverflow occurred for key:", key)

    # Directory doubling if required

    if bucket.depth == self.global_depth:

        self.double_directory()

    # Split bucket

    self.split_bucket(index)

def display(self):

    print("\n--- Extendible Hash Table ---")

    print("Global Depth:", self.global_depth)

    for i, bucket in enumerate(self.directory):

        print(

            "Directory",

            format(i, f"0{self.global_depth}b"),

            "->",

```

```

        bucket.records,

        "(Local Depth:",

        bucket.depth,

        ")"
    )

# Main program

eh = ExtendibleHashing()

records = [1, 3, 5, 7, 9, 11, 13, 15]

print("Inserting 8 records:\n")

for key in records:

    eh.insert(key)

    eh.display()

```

INPUT & OUTPUT:

```

Output
Inserting 8 records:

Inserted: 1

--- Extendible Hash Table ---
Global Depth: 1
Directory 0 -> [] (Local Depth: 1 )
Directory 1 -> [1] (Local Depth: 1 )
Inserted: 3

--- Extendible Hash Table ---
Global Depth: 1
Directory 0 -> [] (Local Depth: 1 )
Directory 1 -> [1, 3] (Local Depth: 1 )

```


Output

Overflow occurred for key: 5

*** Directory Doubled ***

Global Depth: 2

Inserted: 5

--- Extendible Hash Table ---

Global Depth: 2

Directory 00 -> [] (Local Depth: 1)

Directory 01 -> [1, 5] (Local Depth: 2)

Directory 10 -> [] (Local Depth: 1)

Directory 11 -> [3] (Local Depth: 2)

Inserted: 7

Output

--- Extendible Hash Table ---

Global Depth: 2

Directory 00 -> [] (Local Depth: 1)

Directory 01 -> [1, 5] (Local Depth: 2)

Directory 10 -> [] (Local Depth: 1)

Directory 11 -> [3, 7] (Local Depth: 2)

Overflow occurred for key: 9

*** Directory Doubled ***

Global Depth: 3

Inserted: 9

--- Extendible Hash Table ---

Global Depth: 3

Directory 000 -> [] (Local Depth: 1)

Output

--- Extendible Hash Table ---

Global Depth: 3

Directory 000 -> [] (Local Depth: 1)

Directory 001 -> [1, 9] (Local Depth: 3)

Directory 010 -> [] (Local Depth: 1)

Directory 011 -> [3, 7] (Local Depth: 2)

Directory 100 -> [] (Local Depth: 1)

Directory 101 -> [5] (Local Depth: 3)

Directory 110 -> [] (Local Depth: 1)

Directory 111 -> [3, 7] (Local Depth: 2)

Overflow occurred for key: 11

Inserted: 11

```
Output
--- Extendible Hash Table ---
Global Depth: 3
Directory 000 -> [] (Local Depth: 1 )
Directory 001 -> [1, 9] (Local Depth: 3 )
Directory 010 -> [] (Local Depth: 1 )
Directory 011 -> [3, 11] (Local Depth: 3 )
Directory 100 -> [] (Local Depth: 1 )
Directory 101 -> [5] (Local Depth: 3 )
Directory 110 -> [] (Local Depth: 1 )
Directory 111 -> [7] (Local Depth: 3 )
Inserted: 13
```

```
Output
--- Extendible Hash Table ---
Global Depth: 3
Directory 000 -> [] (Local Depth: 1 )
Directory 001 -> [1, 9] (Local Depth: 3 )
Directory 010 -> [] (Local Depth: 1 )
Directory 011 -> [3, 11] (Local Depth: 3 )
Directory 100 -> [] (Local Depth: 1 )
Directory 101 -> [5, 13] (Local Depth: 3 )
Directory 110 -> [] (Local Depth: 1 )
Directory 111 -> [7] (Local Depth: 3 )
Inserted: 15
```

```
Output
Directory 110 -> [] (Local Depth: 1 )
Directory 111 -> [7] (Local Depth: 3 )
Inserted: 15

--- Extendible Hash Table ---
Global Depth: 3
Directory 000 -> [] (Local Depth: 1 )
Directory 001 -> [1, 9] (Local Depth: 3 )
Directory 010 -> [] (Local Depth: 1 )
Directory 011 -> [3, 11] (Local Depth: 3 )
Directory 100 -> [] (Local Depth: 1 )
Directory 101 -> [5, 13] (Local Depth: 3 )
Directory 110 -> [] (Local Depth: 1 )
Directory 111 -> [7, 15] (Local Depth: 3 )

=== Code Execution Successful ===
```

8. Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

CODE:

```
# Secondary Storage Block Access Simulation

# Sequential vs Indexed Retrieval

TOTAL_RECORDS = 100

RECORDS_PER_BLOCK = 10

# Calculate number of blocks

TOTAL_BLOCKS = TOTAL_RECORDS // RECORDS_PER_BLOCK

# Create index

# Index stores: record number -> block number

index = {}

for record in range(1, TOTAL_RECORDS + 1):

    block = (record - 1) // RECORDS_PER_BLOCK + 1

    index[record] = block

# Sequential retrieval

def sequential_retrieval(record_number):

    io_operations = 0

    for block in range(1, TOTAL_BLOCKS + 1):

        io_operations += 1

        start = (block - 1) * RECORDS_PER_BLOCK + 1

        end = block * RECORDS_PER_BLOCK

        if start <= record_number <= end:

            print("Record", record_number, "found in Block", block)

            break
```

```

    return io_operations

# Indexed retrieval
def indexed_retrieval(record_number):

    # One I/O to read index

    index_io = 1

    # Get block number from index

    block = index[record_number]

    # One I/O to read actual data block

    data_io = 1

    print("Record", record_number, "found in Block", block)

    return index_io + data_io

# Main program

record_number = int(input("Enter record number to retrieve (1-100): "))

if record_number < 1 or record_number > TOTAL_RECORDS:

    print("Invalid record number.")

else:

    print("\n--- Sequential Retrieval ---")

    sequential_io = sequential_retrieval(record_number)

    print("Number of I/O operations:", sequential_io)

    print("\n--- Indexed Retrieval ---")

    indexed_io = indexed_retrieval(record_number)

    print("Number of I/O operations:", indexed_io)

    print("\n--- Comparison ---")

    print("Sequential I/O :", sequential_io)

    print("Indexed I/O   :", indexed_io)

```

INPUT & OUTPUT:

```
Output
Enter record number to retrieve (1-100): 87

--- Sequential Retrieval ---
Record 87 found in Block 9
Number of I/O operations: 9

--- Indexed Retrieval ---
Record 87 found in Block 9
Number of I/O operations: 2

--- Comparison ---
Sequential I/O : 9
Indexed I/O    : 2

=== Code Execution Successful ===
```

Show desktop

9. Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.

CODE:

```
import time

import random

import bisect

NUM_RECORDS = 100000

records = [(i, "Student_" + str(i)) for i in range(1, NUM_RECORDS + 1)]

dense_index = [(record[0], i) for i, record in enumerate(records)]

SPARSE_INTERVAL = 100

sparse_index = []

for i in range(0, NUM_RECORDS, SPARSE_INTERVAL):
    sparse_index.append((records[i][0], i))

def dense_search(key):
    keys = [x[0] for x in dense_index]
    pos = bisect.bisect_left(keys, key)
    if pos < len(dense_index) and dense_index[pos][0] == key:
        record_position = dense_index[pos][1]
        return records[record_position]
    return None

def sparse_search(key):
    keys = [x[0] for x in sparse_index]
    # Find nearest index entry <= key
    pos = bisect.bisect_right(keys, key) - 1
```

```

    if pos < 0:
        return None

    start = sparse_index[pos][1]

    # Sequential search within the block
    end = min(start + SPARSE_INTERVAL, NUM_RECORDS)

    for i in range(start, end):
        if records[i][0] == key:
            return records[i]

    return None

SEARCH_COUNT = 1000

search_keys = [
    random.randint(1, NUM_RECORDS)
    for _ in range(SEARCH_COUNT)
]

# Dense index timing
start_time = time.perf_counter()

for key in search_keys:
    dense_search(key)

dense_time = time.perf_counter() - start_time

# Sparse index timing
start_time = time.perf_counter()

for key in search_keys:
    sparse_search(key)

sparse_time = time.perf_counter() - start_time

print("----- Index Comparison -----")

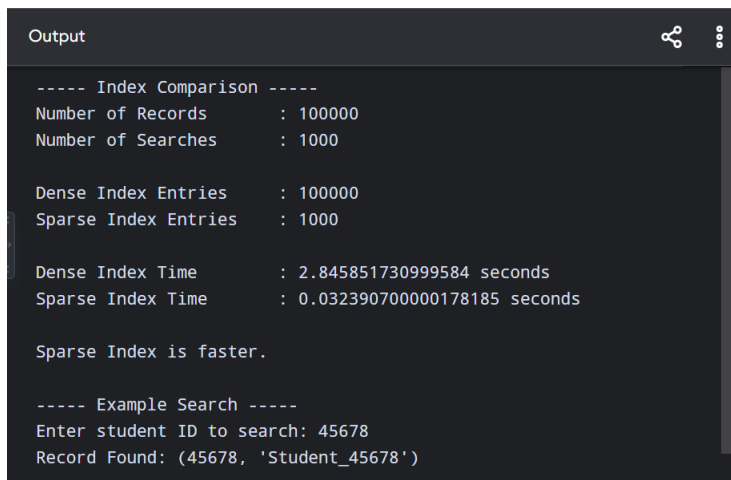
```

```

print("Number of Records    :", NUM_RECORDS)
print("Number of Searches   :", SEARCH_COUNT)
print("\nDense Index Entries   :", len(dense_index))
print("Sparse Index Entries  :", len(sparse_index))
print("\nDense Index Time      :", dense_time, "seconds")
print("Sparse Index Time     :", sparse_time, "seconds")
if dense_time < sparse_time:
    print("\nDense Index is faster.")
else:
    print("\nSparse Index is faster.")
print("\n----- Example Search -----")
key = int(input("Enter student ID to search: "))
result = sparse_search(key)
if result:
    print("Record Found:", result)
else:
    print("Record Not Found")

```

INPUT & OUTPUT:



The screenshot shows a terminal window titled "Output" with a dark background. The output text is as follows:

```

----- Index Comparison -----
Number of Records      : 100000
Number of Searches     : 1000

Dense Index Entries    : 100000
Sparse Index Entries   : 1000

Dense Index Time       : 2.845851730999584 seconds
Sparse Index Time      : 0.032390700000178185 seconds

Sparse Index is faster.

----- Example Search -----
Enter student ID to search: 45678
Record Found: (45678, 'Student_45678')

```


10. Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.

CODE:

```
# B+ Tree with Range Query
```

```
# Range: 15 to 45
```

```
ORDER = 4
```

```
class Node:
```

```
    def __init__(self, leaf=False):
```

```
        self.keys = []
```

```
        self.children = []
```

```
        self.leaf = leaf
```

```
        self.next = None
```

```
class BPlusTree:
```

```
    def __init__(self):
```

```
        self.root = Node(leaf=True)
```

```
    # Insert a key
```

```
    def insert(self, key):
```

```
        root = self.root
```

```
        if len(root.keys) == ORDER - 1:
```

```
            new_root = Node()
```

```
            new_root.children.append(root)
```

```
            self.root = new_root
```

```
            self.split_child(new_root, 0)
```

```
            self.insert_non_full(new_root, key)
```

```
        else:
```

```

        self.insert_non_full(root, key)

# Split a node
def split_child(self, parent, index):
    child = parent.children[index]
    new_node = Node(child.leaf)
    if child.leaf:
        mid = len(child.keys) // 2
        new_node.keys = child.keys[mid:]
        child.keys = child.keys[:mid]
        # Link leaf nodes
        new_node.next = child.next
        child.next = new_node
        parent.keys.insert(index, new_node.keys[0])
        parent.children.insert(index + 1, new_node)
    else:
        mid = len(child.keys) // 2
        middle_key = child.keys[mid]
        new_node.keys = child.keys[mid + 1:]
        child.keys = child.keys[:mid]
        new_node.children = child.children[mid + 1:]
        child.children = child.children[:mid + 1]
        parent.keys.insert(index, middle_key)
        parent.children.insert(index + 1, new_node)

# Insert into non-full node
def insert_non_full(self, node, key):

```

```

if node.leaf:
    node.keys.append(key)
    node.keys.sort()
else:
    i = len(node.keys) - 1
    while i >= 0 and key < node.keys[i]:
        i -= 1
    i += 1
    if len(node.children[i].keys) == ORDER - 1:
        self.split_child(node, i)
        if key >= node.keys[i]:
            i += 1
        self.insert_non_full(node.children[i], key)
# Traverse the tree level by level
def traverse(self):
    print("\n--- B+ Tree Traversal ---")
    queue = [(self.root, 0)]
    while queue:
        node, level = queue.pop(0)
        print("Level", level, ":", node.keys)
        if not node.leaf:
            for child in node.children:
                queue.append((child, level + 1))
# Range query
def range_query(self, start, end):

```

```

node = self.root

# Find first leaf containing start
while not node.leaf:

    i = 0

    while i < len(node.keys) and start >= node.keys[i]:

        i += 1

    node = node.children[i]

result = []

# Follow linked leaf nodes
while node:

    for key in node.keys:

        if key >= start and key <= end:

            result.append(key)

        if key > end:

            return result

    node = node.next

return result

# Display leaf nodes
def display_leaves(self):

    node = self.root

    while not node.leaf:

        node = node.children[0]

    print("\n--- Leaf Nodes ---")

    while node:

        print(node.keys, end=" -> ")

```

```
        node = node.next

    print("NULL")

tree = BPlusTree()

keys = [5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55]

print("Keys inserted:")

print(keys)

for key in keys:

    tree.insert(key)

# Traverse B+ Tree

tree.traverse()

# Display linked leaves

tree.display_leaves()

# Range query

start = 15

end = 45

result = tree.range_query(start, end)

print("\n--- Range Query ---")

print("Range:", start, "to", end)

print("Matching Records:")

for key in result:

    print(key)
```

INPUT & OUTPUT:

```
Output
Keys inserted:
[5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55]

--- B+ Tree Traversal ---
Level 0 : [15, 25, 35]
Level 1 : [10]
Level 1 : [20]
Level 1 : [30]
Level 1 : [40, 45]
Level 2 : [5]
Level 2 : [10]
Level 2 : [15]
Level 2 : [20]
Level 2 : [25]
Level 2 : [30]
Level 2 : [35]

Level 2 : [40]
Level 2 : [45, 50, 55]

--- Leaf Nodes ---
[5] -> [10] -> [15] -> [20] -> [25] -> [30] -> [35] -> [40] -> [45, 50, 55] -> NULL

--- Range Query ---
Range: 15 to 45
Matching Records:
15
20
25
30
35
40
45
```